Advanced Software Requirements Specification (SRS) Restaurant Chain Management System (Zomato-like)

1. Introduction

This document explains the complete requirements of a Restaurant Chain Management System similar to Zomato. The goal of this system is to allow users to order food online, restaurants to manage their menu and orders, and administrators to control the entire platform. This document is written in simple language so that anyone, including beginners, developers, and managers, can understand the system clearly. It describes features, system behavior, architecture, and technical requirements. The system will be built using React with TypeScript and Tailwind CSS for frontend, Node.js with Express for backend, and Supabase PostgreSQL for database management.

2. Overall Description

The system is a web-based application that connects customers, restaurants, and administrators. It acts as a platform where multiple restaurants can register and provide their services. Users can search restaurants, view menus, place orders, and track delivery. Restaurants can manage menus and orders. Admin has full control over users and restaurants. The system is designed to be scalable, secure, and easy to use.

3. System Architecture

The system follows a modern three-tier architecture: 1. Frontend Layer: Built using React and Tailwind CSS. It handles user interface and user interaction. 2. Backend Layer: Built using Node.js and Express. It processes requests and handles business logic. 3. Database Layer: Supabase PostgreSQL stores all data including users, orders, and menus. The frontend communicates with backend APIs, and backend interacts with database securely.

4. User Roles

There are four main roles: 1. Customer: Can browse, order food, and track orders. 2. Restaurant Owner: Can manage menu and orders. 3. Admin: Can manage users and restaurants. 4. Delivery Partner (optional): Handles delivery tracking. Each role has different permissions controlled by role-based authentication.

5. Functional Requirements

User Features: - Register and login securely - Browse restaurants - Search food items - Add items to cart - Place order - Track order - Give ratings and reviews Restaurant Features: - Add/edit/delete menu items - Accept/reject orders - Update order status - View earnings Admin Features: - Manage users - Manage restaurants - View analytics - Block/unblock accounts Order System: - Order status

flow: Pending → Accepted → Preparing → Out for delivery → Delivered

6. Non-Functional Requirements

Performance: The system should handle many users at the same time. Scalability: It should support growth in users and restaurants. Security: Use JWT authentication and secure APIs. Usability: Simple UI with easy navigation. Availability: System should be available 24/7 with minimal downtime.

7. Database Design

Main tables: Users: id, name, email, password, role Restaurants: id, name, location, ownerId Menus: id, restaurantId, itemName, price Orders: id, userId, totalPrice, status OrderItems: id, orderId, menuId, quantity Reviews: id, userId, restaurantId, rating, comment All tables are connected using relationships.

8. API Design

Authentication APIs: - POST /register - POST /login Restaurant APIs: - GET /restaurants - POST /restaurant Order APIs: - POST /order - GET /order/:id Admin APIs: - GET /users - DELETE /user/:id All APIs return JSON responses.

9. Security Requirements

Use JWT tokens for authentication. Encrypt passwords using bcrypt. Validate all inputs. Use HTTPS for secure communication. Implement role-based access control.

10. System Workflow

1. User logs in 2. User selects restaurant 3. User adds items to cart 4. User places order 5. Restaurant accepts order 6. Order is prepared and delivered 7. User gives review

11. Future Enhancements

- AI recommendation system - Live delivery tracking using maps - Payment integration (Razorpay) - Mobile application - Chat system between user and restaurant